src/acquisition.py

Fetching the arXiv corpus and recording provenance

This is the first stage of the pipeline. It queries the public arXiv API for recent cs.CL and cs.LG papers, keeps title, abstract, and metadata for each, and records provenance (query, date, count, SHA-256) so anyone can see exactly what was downloaded. The code ships; the data does not.

Walkthrough of all 230 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Acquire the arXiv corpus and record provenance.
 2
 3  This is the first stage of the RAG pipeline. It queries the public arXiv API for
 4  recent cs.CL / cs.LG papers, keeps title + abstract + metadata for every paper,
 5  and pulls full text only where the paper's licence permits redistribution.
 6
 7  The provenance discipline matches the rest of the portfolio: we record the exact
 8  query, the date, the paper count, and a SHA-256 of the saved bytes, so anyone can
 9  see precisely what was downloaded and when — and detect if it silently changed.
10
11  Nothing here is committed to git; `run_pipeline.py` fetches it and writes
12  `data/raw/`, which is git-ignored. The *code* ships, the data does not.
13  """
```

Lines 1 to 13 are the module docstring. They describe the stage (query arXiv, keep title/abstract/metadata, pull full text only where the licence permits), the provenance discipline (exact query, date, count, and a checksum so silent changes are detectable), and the rule that nothing here is committed to git — the code ships, the raw data stays git-ignored.

```
14
15  from __future__ import annotations
16
17  import hashlib
18  import json
19  import time
20  import urllib.parse
21  import urllib.request
22  import xml.etree.ElementTree as ET
23  from dataclasses import asdict, dataclass
24  from datetime import datetime, timezone
25  from typing import Iterator
26
27  from . import config
```

Lines 14 to 27 are the imports: hashlib for the checksum (line 17), json for JSONL output (line 18), time for polite sleeps (line 19), urllib.parse and urllib.request to build and send API requests (lines 20 to 21), ElementTree aliased ET to parse the XML feed (line 22), asdict and dataclass (line 23), datetime and timezone for the provenance timestamp (line 24), Iterator for a generator return type (line 25), and config (line 27).

```
28
29  # arXiv's Atom API. Documented at https://info.arxiv.org/help/api/index.html.
30  ARXIV_API_URL = "http://export.arxiv.org/api/query"
31
32  # Atom / arXiv XML namespaces, needed to walk the feed with ElementTree.
33  _NS = {
34      "atom": "http://www.w3.org/2005/Atom",
35      "arxiv": "http://arxiv.org/schemas/atom",
36  }
```

Lines 28 to 36 define two module constants. ARXIV_API_URL (line 30) is the Atom API endpoint. _NS (lines 33 to 36) is the namespace map — the Atom and arXiv XML namespaces — needed so ElementTree can find the right tags when walking the feed.

```
37
38  # arXiv licences that permit redistribution of the full text. Papers under these
39  # licences are safe to store and chunk in full; everything else is kept as
40  # metadata + abstract only. This is a deliberate, defensible licensing choice —
41  # not a shortcut — and it is why the corpus is "abstracts for all, full text for
42  # the CC-licensed subset".
43  REDISTRIBUTABLE_LICENSES = (
44      "http://creativecommons.org/licenses/by/4.0/",
45      "http://creativecommons.org/licenses/by-sa/4.0/",
46      "http://creativecommons.org/publicdomain/zero/1.0/",
47  )
```

Lines 37 to 47 define REDISTRIBUTABLE_LICENSES, a tuple of the Creative Commons and public-domain licence URLs under which a paper's full text may be stored and chunked. The comment (lines 38 to 42) frames this as a deliberate licensing choice: full text for the CC-licensed subset, abstracts for everyone else.

```
48
49
50  @dataclass
51  class Paper:
52      """One arXiv paper as we store it.
53
54      `text` is the field retrieval actually runs over: title + abstract always,
55      which is self-contained and licence-safe for every paper. (A future extension
56      could append CC-licensed full text here; the `license` field is recorded so
57      that decision can be made per-paper and defended.)
58      """
```

Lines 48 to 58 open the Paper dataclass — one arXiv paper as stored. The docstring (lines 52 to 58) explains that the text used for retrieval is always title plus abstract (self-contained and licence-safe for every paper), while the recorded license field leaves the door open to appending CC-licensed full text later, decided per paper.

```
59
60      arxiv_id: str
61      title: str
62      abstract: str
63      authors: list[str]
64      categories: list[str]
65      published: str
66      license: str | None
```

Lines 59 to 66 are the Paper fields: arxiv_id, title, abstract, the authors and categories lists, the published date, and license (a string or None, since arXiv only includes a licence when the author selected one).

```
67
68      @property
69      def text(self) -> str:
70          """The retrievable body: title then abstract, one blank line between."""
71          return f"{self.title}\n\n{self.abstract}"
72
73      @property
74      def redistributable(self) -> bool:
75          """True when the paper's licence permits storing its full text."""
76          return self.license in REDISTRIBUTABLE_LICENSES
```

Lines 67 to 76 define two computed properties. text (lines 68 to 71) returns the retrievable body — the title, a blank line, then the abstract. redistributable (lines 73 to 76) returns True when the paper's licence is in the redistributable set, giving a clean per-paper check used when recording provenance.

```
77
78
79  def _build_query_url(start: int, page_size: int) -> str:
80      """Construct one paged arXiv API request URL.
81
82      We search within the configured categories, sorted by submission date
83      (newest first) so the corpus is a coherent recent slice rather than an
84      arbitrary sample — which makes the "recent NLP/ML literature" framing honest.
85      """
86      cat_clause = "+OR+".join(f"cat:{c}" for c in config.ARXIV_CATEGORIES)
87      params = {
88          "search_query": cat_clause,
89          "start": str(start),
90          "max_results": str(page_size),
91          "sortBy": "submittedDate",
92          "sortOrder": "descending",
93      }
94      # arXiv wants the OR-joined category clause un-escaped, so we assemble the
95      # query string by hand rather than letting urlencode escape the '+'.
96      query = "&".join(f"{k}={urllib.parse.quote(v, safe='+:')}" for k, v in params.items())
97      return f"{ARXIV_API_URL}?{query}"
```

Lines 77 to 97 define _build_query_url, which constructs one paged API request. The docstring (lines 80 to 85) notes results are sorted newest-first so the corpus is a coherent recent slice. Line 86 builds the category clause (cs.CL OR cs.LG). Lines 87 to 93 assemble the query parameters (search string, paging offset, page size, sort order). Lines 94 to 96 hand-encode the query string so the OR clause's plus signs are not escaped, and line 97 returns the full URL.

```
 98
 99
100 def _parse_feed(xml_bytes: bytes) -> Iterator[Paper]:
101     """Yield Paper objects from one Atom feed response."""
102     root = ET.fromstring(xml_bytes)
103     for entry in root.findall("atom:entry", _NS):
104         # The arXiv id is the tail of the entry's <id> URL, minus any version.
105         raw_id = entry.findtext("atom:id", default="", namespaces=_NS)
106         arxiv_id = raw_id.rsplit("/", 1)[-1].split("v")[0] if raw_id else ""
107
108         title = (entry.findtext("atom:title", default="", namespaces=_NS) or "").strip()
109         abstract = (entry.findtext("atom:summary", default="", namespaces=_NS) or "").strip()
110         published = entry.findtext("atom:published", default="", namespaces=_NS) or ""
111
112         authors = [
113             (a.findtext("atom:name", default="", namespaces=_NS) or "").strip()
114             for a in entry.findall("atom:author", _NS)
115         ]
116         categories = [
117             c.attrib.get("term", "")
118             for c in entry.findall("atom:category", _NS)
119         ]
120
121         # arxiv:license is present only when the author selected a licence.
122         license_el = entry.find("arxiv:license", _NS)
123         license_url = license_el.text.strip() if license_el is not None and license_el.text else None
124
125         if arxiv_id and title and abstract:
126             yield Paper(
127                 arxiv_id=arxiv_id,
128                 title=" ".join(title.split()),      # collapse arXiv's newlines
129                 abstract=" ".join(abstract.split()),
130                 authors=authors,
131                 categories=categories,
132                 published=published,
133                 license=license_url,
134             )
```

Lines 98 to 134 define _parse_feed, a generator that yields Paper objects from one Atom response. Line 102 parses the XML; line 103 loops over each entry. Lines 105 to 106 extract the arXiv id from the tail of the entry's id URL, stripping any version suffix. Lines 108 to 110 pull the title, abstract, and published date, defaulting to empty strings. Lines 112 to 119 collect the author names and category terms into lists. Lines 121 to 123 read the optional licence element. Line 125 keeps only entries that have an id, title, and abstract, and lines 126 to 134 yield a Paper, collapsing arXiv's internal newlines in the title and abstract with a split-and-join.

```
135
136
137  def fetch_papers(max_papers: int = config.MAX_PAPERS) -> list[Paper]:
138      """Download up to `max_papers` papers from arXiv, paging politely.
139
140      Sleeps between pages to respect arXiv's rate guidance — hammering the API
141      risks a temporary block, which would make the pipeline *less* reproducible,
142      not more. De-duplicates by arXiv id in case paging overlaps.
143      """
144      papers: dict[str, Paper] = {}
145      start = 0
146      while len(papers) < max_papers:
147          url = _build_query_url(start, config.ARXIV_PAGE_SIZE)
148          with urllib.request.urlopen(url, timeout=60) as resp:  # noqa: S310 (trusted host)
149              xml_bytes = resp.read()
150
151          page = list(_parse_feed(xml_bytes))
152          if not page:
153              break  # ran out of results before hitting max_papers
154          for p in page:
155              papers[p.arxiv_id] = p
156
157          start += config.ARXIV_PAGE_SIZE
158          time.sleep(config.ARXIV_REQUEST_DELAY_SECONDS)
159
160      return list(papers.values())[:max_papers]
```

Lines 135 to 160 define fetch_papers, which downloads up to max_papers, paging politely. The docstring (lines 138 to 143) explains the sleeps respect arXiv's rate guidance and that results are de-duplicated by id. Lines 144 to 145 initialise a dict keyed by id and a paging offset. The loop (lines 146 to 158) builds each page URL, fetches and parses it, stops early if a page is empty (lines 152 to 153), stores each paper by id (lines 154 to 155), advances the offset, and sleeps between requests. Line 160 returns the papers, capped at max_papers.

```
161
162
163  def fetch_by_ids(arxiv_ids: tuple[str, ...]) -> list[Paper]:
164      """Fetch specific papers by arXiv id via the API's id_list parameter.
165
166      Used to pull the seed set of seminal papers so the evaluation's ground truth
167      is always present. One request covers all ids.
168      """
169      if not arxiv_ids:
170          return []
171      query = f"{ARXIV_API_URL}?id_list={','.join(arxiv_ids)}&max_results={len(arxiv_ids)}"
172      with urllib.request.urlopen(query, timeout=60) as resp:  # noqa: S310 (trusted host)
173          xml_bytes = resp.read()
174      time.sleep(config.ARXIV_REQUEST_DELAY_SECONDS)
175      return list(_parse_feed(xml_bytes))
```

Lines 161 to 175 define fetch_by_ids, which fetches specific papers by id using the API's id_list parameter — used to pull the seed set of seminal papers so the evaluation's ground truth is always present. Lines 169 to 170 short-circuit on an empty input. Line 171 builds the id_list query; lines 172 to 173 fetch it; line 174 sleeps; line 175 returns the parsed papers. One request covers all ids.

```
176
177
178  def _sha256(path) -> str:
179      """SHA-256 of a file's bytes — the provenance fingerprint."""
180      h = hashlib.sha256()
181      with open(path, "rb") as f:
182          for block in iter(lambda: f.read(1 << 20), b""):
183              h.update(block)
184      return h.hexdigest()
185
```

Lines 176 to 185 define _sha256, the provenance fingerprint. Line 180 creates a SHA-256 hasher; lines 181 to 183 read the file in one-megabyte blocks (the iter-with-sentinel idiom) so a large file is not loaded fully into memory; line 184 returns the hex digest.

```
186
187  def save_papers(papers: list[Paper]) -> None:
188      """Write papers to JSONL and record provenance alongside them."""
189      config.ensure_dirs()
190      with open(config.PAPERS_FILE, "w", encoding="utf-8") as f:
191          for p in papers:
192              f.write(json.dumps(asdict(p), ensure_ascii=False) + "\n")
193
194      provenance = {
195          "source": "arXiv API (http://export.arxiv.org/api/query)",
196          "categories": list(config.ARXIV_CATEGORIES),
197          "downloaded_utc": datetime.now(timezone.utc).isoformat(),
198          "paper_count": len(papers),
199          "redistributable_count": sum(p.redistributable for p in papers),
200          "papers_file": config.PAPERS_FILE.name,
201          "papers_sha256": _sha256(config.PAPERS_FILE),
202      }
203      with open(config.PROVENANCE_FILE, "w", encoding="utf-8") as f:
204          json.dump(provenance, f, indent=2)
```

Lines 186 to 204 define save_papers, which writes the corpus and its provenance. Lines 190 to 192 write one JSON object per paper to the JSONL file (asdict turns each Paper into a dict). Lines 194 to 202 build the provenance record — source, categories, a UTC timestamp, the paper count, how many are redistributable, the filename, and the file's SHA-256 — and lines 203 to 204 write it as indented JSON alongside the data.

```
205
206
207  def load_papers() -> list[Paper]:
208      """Load papers previously saved by `save_papers`."""
209      papers: list[Paper] = []
210      with open(config.PAPERS_FILE, encoding="utf-8") as f:
211          for line in f:
212              papers.append(Paper(**json.loads(line)))
213      return papers
```

Lines 205 to 213 define load_papers, the inverse of save_papers: it reads the JSONL file line by line (lines 210 to 212), reconstructing a Paper from each JSON object, and returns the list.

```
214
215
216 def acquire() -> list[Paper]:
217     """Full acquisition step: fetch recent + seed papers, save, return.
218
219     The seed papers (seminal works the evaluation asks about) are merged in and
220     de-duplicated by id so they are always present regardless of the recent slice.
221     Called by run_pipeline.py.
222     """
223     recent = fetch_papers()
224     seed = fetch_by_ids(config.SEED_PAPER_IDS)
225     merged: dict[str, Paper] = {p.arxiv_id: p for p in recent}
226     for p in seed:                      # seed papers win on id collision
227         merged[p.arxiv_id] = p
228     papers = list(merged.values())
229     save_papers(papers)
230     return papers
```

Lines 214 to 230 define acquire, the full first stage called by run_pipeline.py. The docstring (lines 217 to 222) explains the merge. Line 223 fetches the recent slice; line 224 fetches the seed papers. Line 225 builds a dict of the recent papers keyed by id, and lines 226 to 227 overlay the seed papers so they win on any id collision and are always present. Line 228 flattens back to a list, line 229 saves it with provenance, and line 230 returns it.